LANGUAGE DESIGN / FORMAL MATHEMATICS

Contour

A Human-Scale Proof Language over Lean

Mathematical arguments, bounded synthesis,
explicit obligations, and independently checkable evidence

A design study grounded in

VladimirReshetnikov/ProveIt and
VladimirReshetnikov/Leant

5 September 2026

Central proposal. A mathematician should write the choices that make an argument work. The system should reconstruct routine consequences, display the exact obligations it has reconstructed, and export ordinary Lean evidence. Concision belongs in the source; completeness belongs in the certificate.

Status. This is a language and implementation design, not a report of an implemented compiler. Repository examples were inspected at pinned revisions. The proposed translations have not been compiled in Lean, and no performance or compression benchmark is claimed. Mathematical derivations and the metatheory of a small certificate-assembly core are supplied explicitly.

Abstract

Lean proofs often interleave the mathematical argument with the work needed to express that argument in a particular library: converting intervals, transporting equalities, normalizing casts, instantiating the right lemma, and supplying side conditions. The right response is not to replace formal reasoning with unrestricted prose, nor to put a large opaque prover behind every sentence. This article proposes Contour, a structured mathematical proof language implemented as a layer over Lean. Its basic unit is a typed mathematical step with a fixed conclusion, a permitted dependency context, a method contract, and a checked certificate.

Four source-level case studies from ProveIt motivate the design: binomial inversion, elementary Lambert-W bounds, uniqueness in formal power-series inversion, and a Pascal-tail induction. Worked translations show which information disappears from the primary text, which mathematical choices remain visible, and exactly which obligations the compiler must generate. Leant supplies a particularly relevant architectural lesson: candidate generation, fragment translation, rendering, and verification are distinct boundaries, and failed search must not masquerade as refutation.

The design includes controlled mathematical syntax, scoped inference, certified representation changes, bounded synthesis, explicit induction motives, precise failure states, an expandable proof graph, and search-free replay. A small formal core admits a substitution-based soundness argument. An implementation plan and an adversarial evaluation protocol distinguish a plausible design from unmeasured claims of usability or automation.

Contents

1 The target: shorter arguments, not shorter incantations

The most useful distinction is between a *mathematical decision* and a *reconstructible consequence of that decision.* Choosing an induction parameter, inventing an invariant, introducing the right auxiliary series, or recognizing a binomial identity usually belongs to the first category. Turning membership in a finite interval into two inequalities usually belongs to the second. A good proof language makes the first category prominent and handles the second predictably.

This is not a claim that formal proofs should imitate every habit of informal writing. Human proofs sometimes suppress an essential hypothesis, confuse two meanings of equality, or use a theorem in a setting where it does not apply. The design goal is to preserve the useful compression of mathematical communication without preserving its ambiguity.

The proposed language, provisionally called Contour, has the following organizing rule:

> A source step says *what mathematical fact is established, from what information, and by what kind of reasoning.* An implementation may discover the detailed evidence, but it may not silently change any of those three commitments.

The result is not a second foundational logic. Lean remains the target calculus and the main verification backend. Lean already separates elaboration from kernel checking and supports extension of its syntax and elaborator; those properties make a language layer a realistic starting point [9, 8]. The proposal is primarily a different contract between the author and the elaborator.

1.1 Three artifacts, three different jobs

A completed development should have three synchronized representations. The **argument text** is the maintainable source: intermediate claims, witnesses, transformations, and mathematical explanations. The **obligation graph** records exact types, scopes, side conditions, theorem instantiations, and dependencies. The **certificate graph** contains the elaborated proof terms and their checked environment dependencies.

These are not three independently edited proofs. The latter two are generated from the first and checked against it. A reader can expand a step to inspect its obligations, but a small cast-normalization proof need not occupy ten lines in the primary document. Conversely, a nontrivial lemma cannot disappear merely because an automated search happened to find it.

1.2 Scope of the repository review

The ProveIt observations concern four deliberately selected modules in the FabiusFunction development, not a statistical sample of the entire repository. They span finite combinatorics, inequalities, formal algebra, and induction. The inspected ProveIt revision is `24ce8bd743ea`; the inspected Leant revision is `c36adf114035`. Full revisions, paths, and inspected declaration names appear in Appendix C.

The source inspection establishes what those files contain and how their arguments are organized. It does not establish that every repository theorem is correct, that every selected proof is close to minimal Lean, or that a proposed language would automatically reproduce it. Those distinctions are important: a design study should not present an imagined surface translation as a measured implementation result.

2 What the ProveIt examples actually show

2.1 Binomial inversion: an argument interrupted by representation changes

The module `BinomialInversion.lean` proves the integer-kernel identity

$$\sum_{k=j}^{n}(-1)^{n-k}\binom{n}{k}\binom{k}{j}=\delta_{nj}, \tag{1}$$

then applies a lower-triangular transformation calculus to derive inversion for arbitrary additive commutative groups, with a separate commutative-ring presentation [1]. This is already a good mathematical decomposition. The comment explaining kernel orthogonality is not a substitute for a missing idea; it states the actual idea used by the code.

Inside `sum_Icc_neg_one_pow_choose_mul_choose`, however, the reindexing step includes the following administrative fragment:

```
rw [← Finset.Ico_add_one_right_eq_Icc,
    Finset.sum_Ico_eq_sum_range]
have hlen : j + m + 1 - j = m + 1 := by omega
rw [hlen]
refine Finset.sum_congr rfl fun i _ => ?_
```

The source subsequently proves a natural-number binomial product identity, moves it through casts to integers, and uses `push_cast` and `ring`. The human-level step is: set $k=j+i$, apply

$$\binom{j+m}{j+i}\binom{j+i}{j}=\binom{j+m}{j}\binom{m}{i},$$

and factor out the term independent of i.

A useful language improvement is therefore not an English synonym for `rw`. It is a *certified reindexing operation* that owns the domain bijection, bound arithmetic, and summand transport, together with a coefficient-domain view that owns the casts. The reader should see the substitution and the binomial identity, not the choice between two Mathlib interval encodings.

2.2 Lambert bounds: side conditions are predictable but essential

In `LambertWElementaryBounds.lean`, the decisive inequality is

$$e^{w}\le 1+we^{w}\qquad(w\in\mathbb{R}). \tag{2}$$

The source derives it from the tangent-line inequality $1-w\le e^{-w}$ by multiplying by $e^{w}>0$. Later proofs obtain

$$\frac{x}{1+x}\le W_0(x)\le\log(1+x)\le x\qquad(x\ge 0).$$

The code explicitly establishes positivity before clearing a denominator and interval membership before applying monotonicity [2].

For example, the nonnegativity proof contains:

```
have h0 : (0 : ℝ) ∈ Ici (-Real.exp (-1)) := by
  simp only [mem_Ici]; linarith [Real.exp_pos (-1)]
have hx' : x ∈ Ici (-Real.exp (-1)) := by
  simp only [mem_Ici]; linarith [Real.exp_pos (-1)]
have := principalLambertW_strictMonoOn.monotoneOn h0 hx' hx
simpa [principalLambertW_zero] using this
```

These obligations are real, but they are consequences of $x \geq 0$ and exponential positivity. A language can discharge them behind a monotonicity step while keeping the applicable interval visible in an expandable record. It must not change a restricted monotonicity theorem into a global one.

2.3 Power-series uniqueness: congruence scaffolding obscures composition

The module `LagrangeInversionUniqueness.lean` considers

$$\varphi\psi = 1, \qquad g = X\varphi(g),$$

over an arbitrary commutative ring. It identifies every such g with a canonical solution. Near the end of the principal uniqueness proof, the source constructs several equalities using `congrArg`, then joins them by a nested chain of `trans` applications [3]:

```
have h1 : g = (X : R⟦X⟧).subst g := (subst_X hs).symm
have h2 := congrArg (fun f : R⟦X⟧ => f.subst g) hinv.symm
have h3 := subst_comp_subst_apply hf hs (solution φ ψ hψ)
have h4 := congrArg
  (fun f : R⟦X⟧ => (solution φ ψ hψ).subst f) hcomp
exact h1.trans (h2.trans (h3.trans
  (h4.trans (X_subst (solution φ ψ hψ)))))
```

The conceptual argument is simply

$$g = X \circ g = (s \circ f) \circ g = s \circ (f \circ g) = s \circ X = s.$$

But this notation is legitimate only after verifying the required conditions for formal substitution. The source correctly supplies `HasSubst` evidence. A concise language must preserve that evidence, not treat formal series as arbitrary ordinary functions.

The same module also shows why some seemingly tedious details are mathematically informative. Its cancellation of n in a commutative $\mathbb{Q}$-algebra explicitly proves that $(n : R)$ is a unit. A generic instruction to "cancel a nonzero factor" would be wrong in a ring with zero divisors. This is not bookkeeping to delete; it is a condition whose proof can be automated under a clearly stated algebraic interface.

2.4 Pascal-tail induction: expose the recurrence, hide its plumbing

In `PascalTailIdentity.lean`, define

$$S(j,k) = \sum_{h=0}^{j} \binom{h}{k} 2^{j-h}, \qquad Q(j,k) = \sum_{i=0}^{k} \binom{j+1}{i}.$$

The theorem is $S(j,k) + Q(j,k) = 2^{j+1}$. Its induction step consists of two recurrence relations and elementary arithmetic. One recurrence nevertheless requires extracting $h \leq j$ from finite-range membership before rewriting $j + 1 - h$ [4]:

```
refine Finset.sum_congr rfl fun h hh => ?_
have hle : h ≤ j := Nat.lt_succ_iff.mp (Finset.mem_range.mp hh)
rw [Nat.succ_sub hle, pow_succ]
ring
```

A scoped finite-sum binder should make the bound available automatically. It must still prove the bound: truncated subtraction in $\mathbb{N}$ does not support unrestricted ring identities.

2.5 An important control: some existing Lean is already ideal

Not everything in these files needs a new language. The proof of `le_log_one_add_mul_exp` is already a readable `calc` chain. Reverse binomial orthogonality is a direct application of a previously developed kernel theorem. Several corollaries are single expressions [2, 1].

This establishes the correct baseline. A proposal should be compared not only with the original scripts, but also with carefully refactored Lean using local notation, `calc`, suitable helper lemmas, and existing automation. Otherwise a new syntax can appear impressive merely by taking credit for a better mathematical API. The proposed benefit is a *systematic, scoped, inspectable reconstruction discipline*, not the discovery that helper lemmas make proofs shorter.

3 Precedents and the remaining design space

3.1 Structured and natural-language proof systems

Mizar demonstrates that a structured mathematical document can be the primary proof language, with library organization and built-in inference determining how much a justification must say [15]. **Isar** makes a related distinction between a statically readable proof document and a sequence of procedural commands whose significance depends on replaying proof states [16]. Contour inherits this objective rather than claiming it as new.

Naproche/ForTheL explores controlled mathematical language and automated checking. Its connection to ordinary mathematical prose is particularly relevant, but its logical setting and checking pipeline should not be conflated with Lean's proof-term architecture [17]. The proposed design chooses Lean expressions as the meaning of formulas and requires Lean evidence for completed steps.

Verbose Lean uses controlled natural-language tactics to help students learn traditional proof writing. Its stated educational aim is not simply to minimize keystrokes [18]. **Waterproof** similarly combines a mathematical proof language with an educational editor [19]. Their lessons concern learnable phrasing, logical structure, and feedback; a research-oriented language can adopt those lessons while making denser symbolic calculations its default.

3.2 Sketches, automation, and generated proofs

Wiedijk's formal proof sketches distinguish a formally structured outline from a complete machine-checkable proof [20]. Lamport's hierarchical proofs emphasize the organization of assumptions and subarguments [21]. Contour uses an expandable hierarchy, but a released theorem must have a completed certificate below every formal step. The word "sketch" describes an editing state, not a weaker success criterion.

Aesop supplies configurable, tree-based proof search, including normalization and distinctions between rule classes [22]. Lean's `grind` provides another substantial automation component, combining congruence reasoning with theorem instantiation and specialized solvers [23]. These are potential engines, not competing document formats. A language layer should expose their verified results through stable local interfaces rather than reimplement them without a reason.

Draft, Sketch, and Prove uses informal arguments to guide formal sketch generation and subsequent automated proving [24]. The useful separation is generation versus verification. Contour additionally treats the author's written intermediate conclusions as commitments that a generator may propose to change, but may not silently replace.

3.3 What this proposal adds as a combination

The design contribution is a specific combination: mathematical-step contracts; a dependency-aware elaboration boundary; theorem-backed representation views; explicit handling of semantic ambiguity; Leant-inspired synthesis provenance; and certificate replay independent of search. None of the individual ingredients should be advertised as unprecedented. Their value here is in making the requirements mutually compatible and mapping them to concrete ProveIt examples.

4 The language: write mathematical moves, not proof-state surgery

4.1 A small structural language with mathematical formulas

The name CONTOUR refers to the visible shape of an argument. Its source records the objects introduced, the intermediate assertions, and the transformations that connect them. Below that contour, the elaborator constructs a complete proof.

The structural core should be small. A theorem has a statement and a proof block. Within that block, the author can introduce an arbitrary object, assume a hypothesis, define a local expression, establish a fact, choose a witness, reduce the goal, calculate, distinguish cases, or induct. Formulas remain mathematical expressions, elaborated to Lean terms. These are not arbitrary English sentences sent to a language model for interpretation.

```
theorem composition (A B C : Prop) :
    (A -> B) -> (B -> C) -> A -> C
proof
  assume f : A -> B, g : B -> C, a : A
  conclude C by routine using [f, g, a]
end
```

Here `routine` denotes a configured, bounded reconstruction policy, not an axiom and not a promise that every sensible inference will succeed. One possible certificate is the ordinary Lean term

$$\lambda f\, g\, a.\ g(f(a)).$$

The author may expand the last line into "have $b : B$ from f and a; conclude C from g and b," but need not write that expansion when the application structure is uninteresting.

Long mathematical proofs require more than such propositional plumbing. The essential extension is a library of *mathematical moves*, with a contract for each move. For example, "reindex by $k = j + i$" specifies the intended change of variables; "multiply by e^w" specifies an inequality transformation; "take coefficients" specifies a linear map. The elaborator supplies the relevant congruence lemmas and proves the side conditions.

4.2 What may be omitted

There are four particularly promising classes of omissions.

Logical assembly. Pairing established facts into a conjunction, applying a local implication, transporting an equality through a known function, specializing a universally quantified assertion, and reversing or composing equalities can usually be reconstructed from the immediate neighborhood.

Representation management. A finite interval may appear as `range (n + 1)` or `Icc 0 n`; a ring-valued coefficient may be written as a natural number cast into the ring; a module action on the ring itself may be multiplication. The source should not repeatedly expose the bridge theorems.

Scoped side conditions. A binder $i \in \{0, \dots, n\}$ establishes $i \leq n$. A declared positive real establishes that it is nonzero. A commutative $\mathbb{Q}$-algebra and $n \geq 1$ supply invertibility of the image of n. Such consequences belong in a local obligation service.

Method implementations. A finite-sum reindexing has standard proof obligations. A coefficient calculation has standard linearity rules. The author should select the mathematical operation, not reconstruct its implementation from primitive tactic commands on every occasion.

There is no comparable reason to hide the substitution that makes a proof work, the induction invariant, the choice of a branch, the domain on which a function is injective, or the major theorem on which the argument depends. These are often the proof's mathematical content.

4.3 Statements, reasons, and commentary have different roles

A formal line has three separate components:

$$\underbrace{\text{assertion}}_{\text{what must be proved}} \quad + \quad \underbrace{\text{method and inputs}}_{\text{how reconstruction is constrained}} \quad + \quad \underbrace{\text{commentary}}_{\text{human explanation}} .$$

For example:

```
have h : a*c <= b*c
  by multiply(c) using [a_le_b]
  -- Rescale the inequality to the common denominator.
```

The assertion is formal. The method requests a specific family of proof constructions. The comment does not acquire mathematical authority merely because it sounds plausible. If the method requires $c \geq 0$, that condition becomes an explicit obligation. If it cannot be established, the line remains incomplete.

Method names such as `multiply`, `reindex`, and `coefficients` in this article are proposed interfaces. They are not claims that corresponding general-purpose tactics already exist with these names. Their contract, rather than their spelling, is the design's substance.

4.4 Concision without anonymous context guessing

Anonymous facts are convenient in short calculations, but repeated references to "this," "that," and "the previous result" become fragile in large documents. Internally, each formal assertion receives a stable identity. The author may give it a mathematical name; the editor can otherwise use a source anchor. Reordering unrelated paragraphs should not silently change which fact a later justification denotes.

A missing justification can invoke the local routine policy. A request for broad discovery must be explicit. Thus a local inequality should not unexpectedly trigger a repository-wide search, a network request, or synthesis of a new induction argument. The first may be routinely reconstructible; the latter activities can change both latency and the reader's understanding of the proof.

5 Worked example I: the Lambert inequality

5.1 The mathematical argument, fully exposed

The elementary estimate used in the repository is

$$e^w \le 1 + we^w \qquad (w \in \mathbb{R}). \tag{3}$$

It follows from the tangent-line inequality $1 - w \le e^{-w}$. Multiplication by the positive number e^w gives

$$e^w(1 - w) \le e^w e^{-w} = 1,$$

and rearrangement gives (3). No sign assumption on w is needed. This agrees with the actual signature of the repository theorem, which is stronger than a restriction to $w \ge 0$ would be [2].

Since $e^w > 0$ and $e^w \le 1 + we^w$, both logarithm arguments below are positive. Monotonicity of the logarithm therefore gives

$$w = \log(e^w) \le \log(1 + we^w). \tag{4}$$

For $w \ge 0$, multiplication of (3) by w and division by the positive denominator $1 + we^w$ give

$$\frac{we^w}{1 + we^w} \le w. \tag{5}$$

At $w = 0$, this is simply $0 \le 0$; no cancellation by w is justified or necessary.

Finally, let $x \ge 0$ and $w = W_0(x)$. The defining equation gives $we^w = x$, and the principal-branch monotonicity theorem gives $w \ge 0$. Substituting in (4) and (5), and using $\log(1 + x) \le x$, yields

$$\boxed{\frac{x}{1 + x} \le W_0(x) \le \log(1 + x) \le x.}$$

The branch-domain condition $-e^{-1} \le x$ follows from $x \ge 0$ and $e^{-1} > 0$. It is a generated side condition, not a new hypothesis silently added to the theorem.

5.2 A proposed source representation

The following code assumes a local notation profile for real exponentials and a theorem-backed method profile for ordered-field calculations. The semicolon in a method sequence means that the second stage normalizes the result constructed by the first stage; every stage produces evidence.

```
theorem exp_lambert_bound (w : Real) :
    exp(w) <= 1 + w*exp(w)
proof
  have tangent : 1-w <= exp(-w)
    by apply(exp_tangent(-w))
  have scaled : exp(w)*(1-w) <= 1
    by multiply(exp(w)); normalize using [tangent]
  conclude exp(w) <= 1 + w*exp(w)
    by algebra using [scaled]
end

theorem elementary_bounds (x : Real) (hx : 0 <= x) :
    x/(1+x) <= W0(x) and
    W0(x) <= log(1+x) and log(1+x) <= x
proof
  let w := W0(x)
  have hw : 0 <= w by apply(W0_nonnegative) using [hx]
  have image : w*exp(w) = x by apply(W0_equation)
  have e : exp(w) <= 1+w*exp(w)
    by apply(exp_lambert_bound(w))
  have upper : w <= log(1+w*exp(w))
    by logarithm; normalize using [e]
  have lower : w*exp(w)/(1+w*exp(w)) <= w
    by multiply(w); divide(1+w*exp(w)); normalize
       using [e, hw]
  conclude the stated chain
    by assemble using [lower, upper, image, log_one_add_le(hx)]
end
```

The phrase `the stated chain` denotes the already elaborated theorem target, not a natural-language guess. An implementation could instead use `conclude target`; the meaning is identical. The line beginning `image` is allowed to use a routine proof of the branch-domain condition. It is not allowed to manufacture an assumption asserting that condition.

For this example, the convenience names have fixed resolutions: `exp_tangent` is the inequality supplied by `Real.add_one_le_exp`, normalized at $-w$; `W0_nonnegative` resolves to `Fabius.principalLambertW_nonneg`; `W0_equation` resolves to `Fabius.principalLambertW_mul_exp`; and the final logarithm estimate is obtained from `Real.log_le_sub_one_of_pos`. These are aliases with recorded arguments, not newly assumed mathematical results [2].

5.3 The obligation ledger

An expanded view of the argument should show at least the following obligations:

Mathematical move	Generated obligation	Available evidence
Multiply the tangent bound	$0 \le e^w$	Strict positivity of the exponential.
Normalize the product	$e^w e^{-w} = 1$	Exponential addition, additive cancellation, and $e^0 = 1$.
Apply logarithm	Positivity of the relevant arguments	$e^w > 0$ and the established comparison.
Multiply by w	$w \ge 0$	The local fact h_w.
Divide the inequality	$1 + we^w > 0$	$w \ge 0$ and $e^w > 0$.
Use the branch equation	$-e^{-1} \le x$	$x \ge 0$ and $e^{-1} > 0$.
Assemble the chain	Correct conjunction and inequality directions	The named bounds, substitution of the image equation, and transitivity.

This ledger explains precisely what the shorter source omits. The gain does not come from a black-box assertion that "the Lambert bounds are obvious." It comes from packaging repeatable transformations while leaving the central tangent-line argument visible.

The strict version provides a useful stress test. To obtain strictness in (3), one needs $w \neq 0$ for the strict tangent estimate. To multiply the strict inequality by w without reversing or losing strictness, one needs $w > 0$. A method must not transfer the non-strict proof merely by replacing every $\leq$ with $<$.

6 Worked example II: reindexing and binomial inversion

6.1 The proof a mathematician wants to write

Define the integer-valued kernel

$$K(n,j) = \sum_{k=j}^{n} (-1)^{n-k} \binom{n}{k} \binom{k}{j}, \qquad n, j \in \mathbb{N},$$

where an interval with $n < j$ is empty. We want to show $K(n,j) = \delta_{nj}$.

If $n < j$, both sides are zero. Otherwise write $n = j + m$, where $m \in \mathbb{N}$, and put $k = j + i$. Using

$$\binom{j+m}{j+i}\binom{j+i}{j} = \binom{j+m}{j}\binom{m}{i},$$

we obtain

$$\begin{aligned} K(j+m,j) &= \sum_{i=0}^{m} (-1)^{m-i} \binom{j+m}{j+i}\binom{j+i}{j} \\ &= \binom{j+m}{j} \sum_{i=0}^{m} (-1)^{m-i} \binom{m}{i} \\ &= \begin{cases} 1, & m = 0, \\ 0, & m > 0. \end{cases} \end{aligned}$$

For the last equality, the alternating row sum is 1 when $m = 0$ and 0 when $m > 0$. One may derive it by the binomial theorem, with the $m = 0$ case explicit, or by reflecting the usual alternating-row identity, as the repository does. This proves the desired orthogonality [1].

```
theorem binomial_kernel (n j : Nat) :
    sum(k in [j..n], (-1 : Int)^(n-k)*choose(n,k)*choose(k,j))
      = delta(n,j)
proof
  cases n < j
  case yes:
    conclude target by empty_interval
  case no:
    choose m : Nat with split : n = j+m by arithmetic
    calculate
      sum(k in [j..n], (-1 : Int)^(n-k)*choose(n,k)*choose(k,j))
      = sum(i in [0..m],
          (-1 : Int)^(m-i)*choose(j+m,j+i)*choose(j+i,j))
          by reindex(k = j+i) using [split]
      = choose(j+m,j) * sum(i in [0..m], (-1 : Int)^(m-i)*choose(m,i))
          by sum_algebra using [choose_mul]
      = delta(n,j)
          by cases(m = 0); normalize using [alternating_row, split]
end
```

The notation profile fixes the sums and products in $\mathbb{Z}$. The binomial coefficients are first formed in $\mathbb{N}$ and then mapped into $\mathbb{Z}$. The notation $\delta(n, j)$ denotes the integer conditional "if $n = j$ then 1 else 0." Without these declarations, an apparently familiar formula would leave important semantic choices unresolved.

6.2 What the reindexing operation must certify

For finite sums, a change of variables must preserve both membership and multiplicity. In this example the map

$$\beta : \{i \in \mathbb{N} : 0 \le i \le m\} \longrightarrow \{k \in \mathbb{N} : j \le k \le j + m\}, \qquad \beta(i) = j + i$$

is a bijection. Its inverse on the target interval is $k \mapsto k - j$. The inverse identities use the interval bounds; they are not identities of unrestricted natural-number subtraction.

A `reindex` certificate can consist of this bijection together with a proof that the summands agree at corresponding points. The latter uses

$$j + m - (j + i) = m - i$$

and the chosen interpretation of the coefficient casts. An implementation need not literally construct a subtype equivalence if an existing finite-sum theorem gives a smaller proof. It must, however, establish equivalent obligations.

This is a more substantial abstraction than hiding three `rw` commands behind a macro. The operation knows that the author intends a reindexing, checks its mathematical requirements, and reports a failed bijection or summand equality as such. The bounds introduced by the new summation binder are immediately available to the arithmetic service.

6.3 Cast transport is a theorem-backed view

The identity `Nat.choose_mul` is an equality in $\mathbb{N}$, whereas the desired summand equality lies in $\mathbb{Z}$. The system should first obtain the natural-number identity, apply the canonical homomorphism, and normalize its preservation of multiplication. The proof term can use the same cast lemmas as `push_cast`, but the source need not mention this sequence.

It would be incorrect to treat all casts as syntactic decoration. A map from a ring to a quotient ring preserves addition and multiplication but may not reflect equality. A map into an ordered ring does not automatically support arbitrary order transport. The view declares the direction and the laws it provides. Only those laws are available to reconstruction.

6.4 From orthogonality to inversion

Once the kernel identity is established, the main binomial-inversion theorem is an instance of the repository's lower-triangular transform calculus. For an additive commutative group M, define

$$(Ta)_n = \sum_{k=0}^{n} \binom{n}{k} \bullet a_k, \qquad (Sa)_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} \bullet a_k,$$

with integer scalar action. Orthogonality gives $ST = I$; the existing reverse-orthogonality result gives the other direction. Neither a field nor a $\mathbb{Q}$-algebra is required [1].

A source phrase such as "apply triangular inversion with these kernels" is justified only because that general theorem already exists and its instantiation is displayed. The compiler's task is to connect the pointwise sum notation to the transform representation, not to conceal a newly invented inversion theorem. In an evaluation of concision, the development cost of the transform library must be held constant between Lean and Contour.

7 Worked example III: formal power-series uniqueness

7.1 The decisive mathematical construction

Let R be a commutative ring, and suppose $\varphi, \psi, g \in R[\![X]\!]$ satisfy

$$\varphi\psi = 1, \qquad g = X\varphi(g).$$

Write $f = X\psi$, and let s be the canonical solution constructed in the imported Lagrange-inversion development. That construction supplies

$$s \circ f = X.$$

Here $\circ$ denotes *formal power-series substitution*, not ordinary composition of real functions. The equation for g forces its constant coefficient to vanish; $f = X\psi$ has zero constant coefficient as well. Consequently the substitution operations and the associativity instance used below have the necessary admissibility proofs [3].

Substituting g into $\varphi\psi = 1$ gives $\varphi(g)\psi(g) = 1$, and therefore

$$f \circ g = g\psi(g) = X\varphi(g)\psi(g) = X.$$

It follows that

$$g = X \circ g = (s \circ f) \circ g = s \circ (f \circ g) = s \circ X = s. \tag{6}$$

The mathematical choices are the introduction of $f = X\psi$ and the inverse-composition strategy. The source should retain them. The successive uses of `congrArg` and equality transitivity are implementation details.

```
theorem lagrange_unique
    (R : CommRing) (phi psi g : PowerSeries R)
    (inverse : phi*psi = 1) (equation : g = X*(phi compose g)) :
    g = canonical_solution(phi, psi, inverse)
proof
  let f := X*psi
  let s := canonical_solution(phi, psi, inverse)
  have admissible_g : constant(g) = 0
    by coefficients(0) using [equation]
  have admissible_f : constant(f) = 0 by normalize
  have left : s compose f = X by apply(canonical_inverse)
  have product : (phi compose g)*(psi compose g) = 1
    by substitute(g) using [inverse]
  have right : f compose g = X
    by substitution_algebra using [equation, product]
  calculate
    g = X compose g                          by identity
      = (s compose f) compose g              by rewrite(left)
      = s compose (f compose g)              by associativity
      = s compose X                          by rewrite(right)
      = s                                    by identity
end
```

The bundled-ring binder in this illustration is presentation sugar for a type and a selected commutative-ring instance; it is not a change to Lean's underlying representation. A mechanical lowering must record that instance. Likewise `canonical_inverse` is the specific inverse theorem used by the existing construction, not an unproved assertion that every series has an inverse.

7.2 Why admissibility must not disappear semantically

A generic "composition is associative" rewrite would be the wrong interface. The required theorem has hypotheses about the inner substitution arguments. The `associativity` method in a formal-series profile should synthesize the corresponding `HasSubst` evidence from the two constant-coefficient facts, and attach that evidence to the step.

The source can be shortened further by allowing the formal-series profile to derive those facts on demand. For an introductory or auditing view, displaying them is preferable. This illustrates a useful asymmetry: a fact can be omitted from the default reading view while remaining a named, navigable part of the proof graph.

Formal and analytic power series need separate profiles. The proof above has no radius-of-convergence argument. Converting it to an identity of functions near a point would require an additional bridge theorem and its analytic hypotheses. A change in notation must not perform that bridge implicitly.

7.3 A cancellation example that tests the design

The same module proves a coefficient identity over every commutative ring, with a factor n retained:

$$n[X^n]\big(H\varphi^{n-1}(\varphi - X\varphi')\big) = [X^{n-1}]\big(H'\varphi^n\big), \qquad n \geq 1. \tag{7}$$

Its calculation differentiates $H\varphi^n$, takes coefficients, and rearranges. The characteristic-free form matters. The subsequent $\mathbb{Q}$-algebra theorem cancels n by proving that its image in R is a unit [3].

A human-oriented command "cancel n" must request a cancellation certificate in the actual ring. Knowing $n \geq 1$ in $\mathbb{N}$ is insufficient in an arbitrary ring. Even a nonzero element can fail cancellation:

in $\mathbb{Z}/6\mathbb{Z}$, $2 \cdot 0 = 2 \cdot 3$ although $0 \neq 3$. In a $\mathbb{Q}$-algebra, the inverse of the image of n is supplied by the image of $1/n$. The elaborator can reconstruct this standard argument; it cannot replace it with the slogan that positive integers are nonzero.

This also shows why adding stronger hypotheses is not an acceptable automatic repair. Requiring R to be a field would make many steps easier but would weaken the author's theorem. The interface must instead show the unresolved cancellation requirement or suggest a separate, explicitly stated specialization.

8 Worked example IV: induction on a finite-sum identity

For $j, k \in \mathbb{N}$, put

$$S(j,k) = \sum_{h=0}^{j} \binom{h}{k} 2^{j-h}, \qquad Q(j,k) = \sum_{i=0}^{k} \binom{j+1}{i}.$$

The repository proves the additive identity

$$S(j,k) + Q(j,k) = 2^{j+1}. \tag{8}$$

Two recurrence relations carry the induction:

$$S(j+1,k) = 2S(j,k) + \binom{j+1}{k}, \tag{9}$$

$$Q(j+1,k) + \binom{j+1}{k} = 2Q(j,k). \tag{10}$$

The first splits the final term from the sum and uses $h \leq j$ to rewrite $j + 1 - h = (j - h) + 1$. The second is the summed Pascal rule with its two boundary terms accounted for [4].

At $j = 0$, if $k = 0$, then $S(0,0) = 1$ and $Q(0,0) = 1$. If $k > 0$, then $\binom{0}{k} = 0$ and the only nonzero terms of $Q(0,k)$ occur at $i = 0, 1$, giving $Q(0,k) = 2$. Thus the base case is valid for every k.

For the induction step, let $c = \binom{j+1}{k}$. The recurrence relations and induction hypothesis give

$$S(j+1,k) + Q(j+1,k) = 2S(j,k) + c + Q(j+1,k) = 2\big(S(j,k) + Q(j,k)\big) = 2^{j+2}.$$

Notice that this argument uses no subtraction in $\mathbb{N}$. The subtraction form of the theorem is a consequence of the additive one, not the easiest starting point.

```
theorem pascal_tail (j k : Nat) : S(j,k)+Q(j,k) = 2^(j+1)
proof
  induct j keeping k
  case zero:
    cases k = 0
    case yes: conclude target by finite_sum_normalize
    case no:  conclude target by finite_sum_support(choose(1,_))
  case successor j ih:
    have s : S(j+1,k) = 2*S(j,k)+choose(j+1,k)
      by split_last; sum_algebra
    have q : Q(j+1,k)+choose(j+1,k) = 2*Q(j,k)
      by apply(tail_pascal_recurrence)
    conclude target by arithmetic using [s, q, ih]
end
```

The support method in the base case is not unrestricted enumeration up to the symbolic bound k. It uses the theorem that $\binom{1}{i} = 0$ for $i > 1$, isolates the possible nonzero terms, and uses $k > 0$ to

establish that 1 lies in the range. Similarly, `split_last` needs a summation profile that unfolds the local definition of S and records the new bound on the retained index.

The induction motive here is exactly

$$\lambda j.\ S(j,k) + Q(j,k) = 2^{j+1},$$

with k fixed. It is displayed by the editor even when its lambda notation is absent from the source. Other proofs genuinely require generalizing parameters before induction. The language should support an explicit `generalizing` clause, and a suggestion engine may recommend one; it must not hide a change of induction hypothesis behind a vague claim of "the same argument."

9 Elaboration and a small formal core

9.1 Fix the meaning before searching for a proof

A proof-producing system can still prove the wrong statement. For example, the expression $a - b$ can denote natural subtraction, integer subtraction, or subtraction in a chosen ring. A notation for composition can denote function composition or formal substitution. A quantified statement can differ drastically when the order of $\forall$ and $\exists$ is changed.

The elaborator should therefore separate *statement elaboration* from *proof reconstruction*. Statement elaboration resolves binders, namespaces, universes, coercions, structures, and notation into a particular Lean expression. Reconstruction then receives that expression as a fixed target.

This is not a prohibition on ordinary type inference. Expected types and declared local notation may determine what a numeral or overloaded operation means. Nor does it prohibit proof metavariables needed to construct a term. The prohibited operation is using the availability of an easy proof to choose among materially different theorem statements without exposing that choice.

An unresolved object type should prompt a precise diagnostic, such as "the subtraction domain is not determined: $\mathbb{N}$, $\mathbb{Z}$, or R." A language-model assistant may propose the intended choice, but adopting that choice is an edit to the formal statement. It is not merely an invisible step of proving it. Free identifiers must not silently turn into new hypotheses that make the theorem easier.

9.2 The elaboration pipeline

A practical pipeline has six stages:

1. Parse the document structure and its mathematical expressions, retaining source spans and stable assertion identities.
2. Elaborate and freeze the theorem statement and declared notation profile in a known Lean environment.
3. Lower proof blocks into a scoped graph of typed obligations and explicit structural proof constructors.
4. Reconstruct local evidence using the selected method profiles, theorem retrieval, or a synthesis adapter.
5. Assemble the evidence, check it against the frozen statement, and audit its dependencies and assumptions.
6. Export a replayable certificate together with an explanation map from source steps to obligations and evidence.

The compiler should preserve Lean's elaborated expressions rather than repeatedly translating them through pretty-printed strings. Strings are useful for display and interoperability, but do not by themselves preserve binder identity, universe constraints, selected type-class instances, or the identity of overloaded constants. Lean's existing elaboration and metaprogramming facilities supply the appropriate starting point [10, 8].

9.3 An obligation is more than a proposition

A proposed obligation record is

$$O = (\iota, \mathcal{E}, \Gamma, T, M, D, B, S),$$

where ι is a stable identity, $\mathcal{E}$ the fixed global environment, Γ the ordered local context, T the exact target, M the permitted method, D the dependency policy, B the resource budget, and S the source location and explanation metadata.

The order in Γ matters. A hypothesis may mention an earlier object; a vector may have a type depending on an earlier length; a proposition may mention an earlier witness. Context selection must preserve this dependency closure. Choosing a few apparently relevant hypotheses as disconnected strings is insufficient.

The dependency policy also has two layers. A *mathematical input policy* says which local facts and named theorem interfaces may justify the step. A *foundation policy* says which transitive axioms and trusted declarations may occur in the resulting evidence. The latter cannot simply forbid every declaration not named in the source: any useful imported theorem has a substantial transitive implementation. Conversely, a method's permission to use a named theorem does not justify introducing a new axiom.

9.4 Structural commands and their proof constructors

The following rules describe the intended core. They are specifications for lowering, not additional inference rules trusted by Lean.

Arbitrary object and assumption. To prove $\forall x : A, P(x)$, `take x : A` opens a fresh binder and proves $P(x)$. The result is a lambda. To prove $P \to Q$, `assume h : P` opens a proof binder and proves Q. An `assume` command cannot simply add a convenient undischarged premise to an unrelated goal.

Intermediate assertion. If a block constructs $p : P$ and its continuation constructs $q : Q$ in context $h : P$, then

$$\texttt{have}\ h : P\ \texttt{by}\ p;\ q \quad \longmapsto \quad \texttt{let}\ h : P := p\ \texttt{in}\ q.$$

The continuation can mention h while editing, but completion requires the evidence p. An unfilled assertion is not exported as an assumed theorem.

Reduction. A command `suffices Q` while proving P requires both a proof of Q and a proof of $Q \to P$ in the applicable context. The connection may be reconstructed routinely, but it must exist. Reduction is not permission to replace the target with a more convenient proposition.

Witness. To prove $\exists x : A, P(x)$, a witness command produces a term $a : A$ and a proof of $P(a)$, lowered through existential introduction. To use an established existential, the new witness is scoped under existential elimination. It must not escape into an arbitrary computational result. Lean's proof-irrelevant `Prop` has restricted elimination into data types; a constructive data result should instead use a dependent pair or subtype, or an explicit classical-choice construction under the selected axiom policy [11].

Cases. Case blocks must be exhaustive with respect to a checked eliminator or a proved disjunction. For a split on an arbitrary proposition, classical or decidable case analysis must be accounted for. A branch assumption is local to its branch.

Calculation. Each edge in a calculation is a separately typed relation. Equality transitivity assembles equalities; an order chain needs the appropriate mixed transitivity rules. A chain containing $a < b \leq c$ must establish $a < c$, not merely $a \leq c$. A registered relation profile supplies the legal composition theorems.

Induction. The induction command elaborates a motive, supplies all constructor cases, and lowers to the relevant recursor or a previously proved induction principle. Generalized parameters are explicit in the motive. This follows Lean's treatment of induction and recursion through recursors, rather than introducing an independent trust mechanism [12].

9.5 A certificate-assembly theorem

A small mathematical model makes the trust boundary precise. Let $\mathcal{E}$ be a well-formed environment and Γ a well-formed context. Suppose an elaborated skeleton s has type T in the extended dependent context

$$\mathcal{E}; \Gamma, z_1 : O_1, z_2 : O_2(z_1), \ldots, z_m : O_m(z_1, \ldots, z_{m-1}) \vdash s : T.$$

The z_i are *placeholders*, not new axioms in the released environment. They can stand for proofs or for typed witness data. The frozen exported target T contains none of these placeholders.

Theorem 9.1 (Soundness of acyclic certificate assembly)**.** *Suppose, for each i, there is evidence d_i such that*

$$\mathcal{E}; \Gamma, z_1 : O_1, \ldots, z_{i-1} : O_{i-1} \vdash d_i : O_i,$$

with no unresolved metavariables or additional free identifiers. Define

$$e_1 = d_1, \qquad e_i = d_i[e_1/z_1, \ldots, e_{i-1}/z_{i-1}].$$

Then the assembled expression

$$p = s[e_1/z_1, \ldots, e_m/z_m]$$

has type T in $\mathcal{E}; \Gamma$.

Proof. Proceed through the telescope in order. For $i = 1$, the first typing judgment already gives $e_1 : O_1$ in $\mathcal{E}; \Gamma$. Assume the previous evidence terms have the types obtained by substituting their predecessors. The dependent substitution lemma applied successively to the judgment for d_i gives

$$\mathcal{E}; \Gamma \vdash e_i : O_i[e_1/z_1, \ldots, e_{i-1}/z_{i-1}].$$

The same successive substitutions in the judgment for s give a term whose type is the corresponding substitution into T. Since T contains no placeholders, that type is exactly T. All intermediate contexts are well formed by the typing assumptions and the order of the telescope. □

Nested proof blocks are represented using their binders and eliminators before applying this model; equivalently, their obligations can be closed over the ordered local context. It would be unsound to flatten a branch-local witness into an unrestricted global constant. The theorem assumes well-scoped lowering precisely to exclude that mistake.

The result gives a simple engineering principle: a search engine may be incomplete, inefficient, or wrong about a candidate, but successful assembly requires actual evidence at every node. Search failure does not affect soundness; accepting an unchecked node does.

This theorem is a paper proof about the specified core. It is not a mechanized verification of a parser, a Lean implementation, or a mapping from unrestricted human language to formulas. The production acceptance boundary should still recheck the assembled expression against the separately frozen target.

9.6 Cycles and shared unknowns

Mutually dependent claims cannot be accepted merely because each is derivable from the other. The graph must be acyclic after accounting for explicit recursors, well-founded arguments, or simultaneous-induction principles. Those principles supply the justification for recurrence; graph cycles alone do not.

Some obligations share unknown data. In proving $\exists x, P(x) \wedge Q(x)$, the two conjuncts must concern the same x. It is incorrect to solve them independently with unrelated witnesses and merge the apparent successes. The implementation should form dependency components around shared metavariables and solve each component transactionally. Failed alternatives restore the complete metavariable and local-instance state. Parallel search is safe only with appropriate isolation and a checked merge of compatible assignments.

10 Leant integration and the synthesis boundary

10.1 A directly relevant architecture

Leant offers two complementary mechanisms. Its interactive proving interface can suggest tactic scripts and test whether they close the goal. Its `:synth` interface constructs inhabitants of a requested type through Djex-backed search, then re-elaborates proposed terms against the Lean goal. The README's recursive `double` example illustrates a suggested combination of induction, simplification, and arithmetic; the synthesis examples illustrate composition of local functions and logical evidence [5].

These are useful ingredients, but their strongest lesson is architectural. The proposed language should distinguish suggesting a proof, searching a fragment, rendering a candidate, checking the original target, and recording the accepted evidence. A search result is not authoritative merely because its internal representation was well typed.

The inspected `Fragment.hs` explicitly decomposes structural connectives and certain inductive forms while retaining other expressions as opaque atoms. Proper-type application structure is treated differently from term-indexed applications, and the translator records safety information relevant to negative verdicts. A dependent formula carried as an atom can participate in implication or product assembly without its internal mathematics being analyzed [6].

This is exactly the right division of labor for many local gaps in Contour. If an obligation requires combining $h_1 : P \to Q$, $h_2 : P$, and $h_3 : Q \to R$, the structural engine can supply the composition even when P, Q, R contain sophisticated mathematics. If the missing step is a new analytic estimate inside P, treating P as an atom will not discover that estimate.

10.2 An obligation-to-synthesis interface

A proposed adapter can use the following conceptual types. These are interface specifications, not names of existing Leant APIs.

```
SynthesisRequest = {
  obligationId, frozenEnvironment, orderedLocalContext,
  exactTarget, allowedProviders, methodPolicy, budget
}

Candidate = {
  originatingRequest, proposedTermOrSketch,
  providerInstantiations, originEvidence, estimatedCost
}

CheckedCompletion = {
  originatingRequest, exactLeanTerm, exactLeanType,
  actualDependencies, axiomInventory, replayArtifact
}
```

The request's target remains a Lean expression in the host. The adapter may translate a supported projection for a search engine, but it retains the association between that projection and the original obligation. Only the Lean-side acceptance service can create a `CheckedCompletion`; the search engine can create candidates.

The detailed Leant design preserves a semantic-origin record and exact provider assignments, and treats the candidate together with its associated evidence as a unit. It also warns that changes such as wrapping a term in a classical proof construction produce a new term and do not inherit the original candidate's authority automatically [7]. Contour should retain the same discipline: an apparently identical printed term is not a license to attach evidence from a different run or environment.

For an initial implementation, this does not require rewriting Leant in Lean. A Lean-side obligation service can communicate with the existing backend and keep exact-target checking on the Lean side. A later in-process implementation may reduce serialization and elaboration overhead. Either arrangement must preserve the same acceptance contract.

10.3 Positive and negative results have different requirements

A positive candidate has a straightforward acceptance test: reconstruct a term and check it at the exact original type. Translation may lose opportunities without compromising accepted results.

A negative answer is different. An unsuccessful search over an abstraction does not establish non-inhabitation of the original type. Even exhaustive search over a restricted grammar proves, at most, that no term in that grammar was found. Leant's distinction between complete-fragment negative results and bounded inconclusive results is therefore essential [5, 6].

For the proposed language, the visible result states should include `proved`, `unsupported`, `budget exhausted`, `timed out`, and `candidate rejected`. A claim that the mathematical goal is refuted requires a checked proof of its negation, or a separately justified lifting of a certified refutation from an exact fragment. Without that bridge, even a useful fragment-level impossibility result must retain its narrower label.

Counterexamples obtained by testing or an external solver are valuable diagnostics and search guidance. A refutation of a universal theorem can sometimes be certified by one concrete witness, but the witness and its violated predicate must then be checked in the original semantics. Raw solver output is not itself the proof object.

10.4 Synthesize witnesses and their specifications together

Leant's README contains an instructive state-threading example: more than one term has the type of a state-monad bind, and a type-correct candidate can pass the initial state rather than the updated state. A constant `none` is likewise a legitimate inhabitant of some option-processing types without being the intended operation [5].

The analogous issue arises in mathematical witness construction. Searching only for $x : A$ and then attempting to prove $P(x)$ can commit to an unhelpful witness. Prefer a joint request such as

$$\{x : A \mid P(x)\} \quad \text{or} \quad \sum_{x:A} B(x),$$

when data is required, or a proof of $\exists x : A, P(x)$ when only a proposition is required. The latter does not automatically provide a computational extractor; the distinction follows the core's witness rules.

For the binomial proof, the request is not merely "find a natural number m." It is "find m with $n = j+m$, under $j \leq n$." A linear-arithmetic witness method can choose $n - j$ and prove its specification in one operation. For a limit proof, a witness N must depend only on variables in scope, including the previously chosen ε, not on a later universally quantified index.

Proof irrelevance does not make this issue disappear. Proofs of a fixed proposition are irrelevant to Lean's computation, but the data selected in a dependent pair can be observably different. Nor does proof irrelevance imply that two explanations of a theorem are equally useful to a reader [11].

10.5 Rank for an argument, not just for a small term

Candidate ranking should consider several costs: distance from the author's selected method, number of new mathematical dependencies, certificate size, checking cost, and sensitivity to surrounding library changes. Source length is only one secondary signal. The smallest term may be a call to an enormous theorem that hides the intended argument; a slightly larger term may express exactly the local calculation the author meant.

A useful ordering is lexicographic rather than an opaque score: reject semantic or dependency violations first; prefer the requested method and local facts next; compare verification and presentation costs only among compliant candidates. Search budgets and ranking profiles must be recorded separately from accepted evidence. The inspected Leant internals already distinguish ranking and candidate-window policy from later verification and origin-preserving handoffs [7].

An external prover or language model may suggest a stronger lemma, a different induction parameter, or a new representation. Such suggestions belong in the editing interface. They become part of the formal argument only when adopted as visible changes, not when smuggled into a routine completion.

11 A library of mathematical moves

11.1 Contracts rather than unrestricted tactic aliases

A method profile should specify its input shape, the obligations it may introduce, its permitted background lemmas, its evidence constructor, and its normalization policy. For example, a finite-sum reindexing profile recognizes a source domain, target domain, and map; requests membership, inverse or injectivity/surjectivity evidence as appropriate; and proves summand agreement. It is not simply an arbitrary tactic hidden under an English name.

The distinction matters when explaining success. An unrestricted search might prove the requested sum equality by an unrelated closed-form theorem. That is mathematically valid but does not certify the assertion that the step was a reindexing. A contract-bound method can retain a structured certificate whose root is the reindexing theorem and whose children discharge precisely its conditions.

Not every proof should be forced into such a contract. An explicit `by search` or local Lean escape block may allow a more general derivation, with the dependencies displayed. The source then says what actually happened rather than claiming a specific method that was never used.

11.2 Certified representation views

A view connects two presentations through a theorem. In the simplest case it supplies $u = v$ and uses equality transport. More generally, a relation $V(a, b)$ relates representations, together with a transport theorem

$$V(a, b) \to P(a) \to Q(b).$$

The direction is explicit. An isomorphism can supply stronger transport than a non-injective homomorphism, and an implication supplies less than an equivalence.

Useful initial views for the reviewed corpus include finite intervals versus ranges; integer scalar action versus multiplication in a ring; function equality versus pointwise equality; natural or integer coefficient casts; and coefficient notation versus the corresponding linear maps. Each view is local to a profile and versioned with its supporting theorems.

The elaborator must distinguish four notions that informal notation often merges: definitional equality, proved equality, equivalence of propositions, and equivalence of structures. Definitional equality is handled by conversion. A proved equality requires a transport term. A logical equivalence is not automatically an equality without the appropriate principle. An isomorphism between structures is not permission to identify their inhabitants without a specified map. Dependent goals make these differences particularly important.

A view search should have a bounded depth and a canonical orientation where possible. Unrestricted alternation between equivalent presentations can loop, inflate terms, or obscure why a theorem applies. The selected route is saved in the certificate so replay does not rediscover it.

11.3 A scoped side-condition service

The service should maintain derived facts with provenance, not a bag of untyped predicates. A positive term can supply nonnegativity and nonzeroness. Membership in a finite interval supplies its bounds. A unit supplies a nonzero fact only under suitable nontriviality hypotheses, while cancellation by a unit needs no such detour. The service should ask for exactly the property required by the method rather than indiscriminately deriving every plausible consequence.

Facts have lexical scope. The bound $h \leq j$ arising inside a summand proof cannot be exported as a fact about an arbitrary h outside that binder. Likewise a positivity assumption from one case cannot leak to its sibling. Every derived fact carries its context and proof.

For totalized operations, a well-typed expression need not justify a familiar transformation. Lean's natural subtraction and division conventions do not make unrestricted cancellation or subtraction identities valid [14]. The side-condition service belongs to the theorem-backed transformation layer, not to parsing. Thus writing a/b is possible when $b = 0$, but clearing that denominator using an ordinary field identity requests $b \neq 0$.

11.4 “Without loss of generality” as a checked reduction

A useful general schema for a normalization argument is the following. Let a family of transformations g act on a domain A, let P be the desired property, and let Q specify a preferred representative. Suppose we have

$$\forall x \in A\ \exists g,\ Q(gx), \qquad \forall x, g,\ P(gx) \to P(x), \qquad \forall y \in A,\ Q(y) \to P(y),$$

with closure of the transformations in A included. Given x, choose g from the first assertion, apply the third to gx, and transport the result back with the second. This proves $P(x)$.

A `wlog` block can implement this schema, but it must request both coverage and transport evidence. For example, to assume $x \le y$ in a theorem symmetric in two real variables, the transformations are identity and swap. Totality of the order gives coverage; symmetry gives transport. Without symmetry, the reduction is invalid. To normalize a nonzero vector to unit length, additional scalar and norm hypotheses arise. A generic “by symmetry” slogan cannot discharge them all.

For an initial release, this construct should accept an explicit transformation and reduction theorem. Automatic discovery of the right symmetry group can remain a suggestion feature. That keeps a powerful mathematical abbreviation from becoming an opaque proof search in disguise.

11.5 “Similarly” means instantiate and recheck

A proof template can be reused with a substitution of parameters, operations, or structures. Its substitution must be hygienic and type directed, and all resulting obligations must be checked again. The compiler must not copy an earlier success flag or assume that a textual replacement preserves hypotheses.

The strict Lambert example is a good test. Replacing a non-strict inequality with a strict one changes the condition for multiplication by w. A template can reveal the new requirement $w > 0$ and reuse the remaining structure. It cannot suppress that difference in the name of similarity.

For long developments, a checked proof template can be exported as a named method or ordinary lemma. The choice depends on whether the reusable object is a theorem with fixed semantics or an elaboration pattern that still generates substantive obligations.

11.6 Analytic moves should be conservative at first

Phrases such as “take limits,” “differentiate,” and “interchange sum and integral” are attractive because they resemble ordinary mathematical prose. They are also where omitted hypotheses often carry the real difficulty.

A limit method needs the exact filters or convergence notions, convergence proofs for its inputs, and the appropriate continuity theorem at the relevant point. A substitution inside an integral may require measurability, integrability, domain information, and a change-of-variables theorem. Almost-everywhere equality is not pointwise equality. Differentiability at a point is not automatically differentiability on a neighborhood.

These are good eventual method profiles, but a first implementation should demand named supporting theorems rather than attempting to infer a complete analytic setting from prose. The finite-sum and ordered-field profiles already exercise the core architecture without requiring an overly ambitious universal notion of routine mathematics.

12 An interface for reasoning, not just for checking

12.1 Forward reasoning and backward planning

Mathematical work often alternates between consequences of the assumptions and conditions sufficient for the goal. A document language should permit both without forcing the author to keep an imperative tactic stack in mind.

A forward step records a fact established from earlier information. A backward step records a sufficient subgoal and its connection to the current goal. The interface can display these as two frontiers in one argument graph: facts available below, obligations pending above, and checked links between them. A line closes the gap only when the evidence connects the frontiers.

For the formal-series example, the forward frontier contains $\varphi\psi = 1$ and $g = X\varphi(g)$. A backward plan says that it would suffice to establish $f \circ g = X$ for a known left inverse s of f. Introducing $f = X\psi$ is the mathematical insight connecting the two. The editor should display that plan, not only the low-level sequence of equality rewrites eventually used to realize it.

12.2 Discovery and the final proof are different documents

A research notebook can contain conjectures, failed approaches, candidate witnesses, numerical experiments, and temporary assumptions. Those are useful records of reasoning, but they are not established facts. The interface should distinguish a scratch assertion from a proved assertion visually and semantically.

An exploratory block may temporarily assume a conjecture to investigate its consequences. Exporting a theorem from that block must either discharge the conjecture, expose it as an explicit hypothesis in a separately named conditional result, or remain incomplete. It must not silently promote the conjecture to a library theorem. The author can preserve the exploratory narrative without contaminating the released theorem's assumptions.

A synthesis suggestion can be shown at different scales: a term that fills a local gap; a tactic sequence that realizes a chosen step; a proposed lemma; or a revised proof plan. These are different kinds of edits and should not share a single undifferentiated "accept" operation. A term completion preserves the statement. A lemma proposal adds mathematical content. A revised plan changes the visible argument.

This distinction supports actual discovery without pretending that the final proof is a transcript of the author's mental activity. "Human-scale" means that the language exposes recognizable mathematical decisions, not that it models a universal psychology of mathematical thought.

12.3 Expandable explanations

The default view should display the source-level argument. Expanding a line should show its exact target, the selected theorem instances, automatically established side conditions, and the reason each obligation was generated. A further expansion can show Lean evidence or a corresponding explicit proof script.

For the Lambert division step, an informative expansion says: "division preserves this order because $1 + we^w > 0$; positivity follows from $w \geq 0$ and $e^w > 0$." The underlying term is available, but the first explanation should remain mathematical. For the binomial reindexing, the useful expansion shows the map, its inverse on the two intervals, and the summand identity.

The displayed explanation should be generated from the accepted proof structure and method certificate, not improvised independently after success. Otherwise the system could produce a correct proof with

an incorrect account of why it works. Free-form commentary may still be generated as a suggestion, but it should not masquerade as a checked derivation.

12.4 Failure should identify the missing mathematics

A failed method should report its remaining obligations, not just the last low-level tactic error. Examples include:

```
Cannot clear this denominator.
Required: 0 < d
Known:    d != 0
The sign of d is not determined; split on its sign or use another method.

Cannot apply the substitution-associativity rule.
Required: HasSubst g
Available facts do not establish the required substitution condition.

Cannot finish this induction step with the selected hypothesis.
Current motive keeps parameter k fixed.
A proposed generalization over k is available as a proof-plan edit.
```

These are schematic diagnostics. They express what the user needs to know without claiming that the requested theorem is false. A timeout should display the attempted method and remaining goal; it should not relabel a difficult obligation as invalid mathematics.

12.5 Three distinct notions of success

A complete interface should distinguish:

1. **Logical success:** the certificate has the frozen formal type under the declared assumptions.
2. **Statement fidelity:** that formal type and its definitions express the author's intended mathematical claim.
3. **Explanatory fidelity:** the displayed argument accurately describes the accepted derivation at the chosen level of detail.

Kernel checking establishes the first relative to its environment and logic. It does not, by itself, establish the second or the third. Statement previews, explicit notation profiles, constrained method certificates, and dependency displays support those additional checks. They do not eliminate the need to read the theorem and understand its definitions.

There is also a limit to mechanical dependency claims. A proof term may syntactically mention a lemma that is mathematically redundant. Proof irrelevance and deliberate padding make "this premise was genuinely necessary" a stronger claim than ordinary dependency analysis can justify. The proposed system records actual references and checks method structure; it does not claim to solve semantic necessity or to certify the best explanation.

13 Trust, certificates, and reproducible replay

13.1 The release boundary

A theorem is releasable only when all obligations are discharged, all local binders are properly closed, the assembled term has the frozen type, and its transitive assumptions satisfy the selected policy. A

proof that relies on a missing dependency remains incomplete even if the final source file contains no visible hole.

The policy must inspect assumptions transitively, including those occurring in dependencies. It should reject unresolved metavariables, placeholders, and `sorryAx`; flag any new axioms; and report its permitted foundational assumptions. A classical mathematics profile can allow the standard choice, propositional-extensionality, and quotient assumptions where used. A constructive profile can be stricter. Lean's official validation guidance explicitly distinguishes ordinary success indicators, transitive axiom inspection, and stronger replay checks [13].

The language implementation and search workers need not become part of the logical trusted base merely because they generate terms. However, a deployment that executes arbitrary metaprograms in the same process as the checker does not obtain an independent verification boundary solely by naming it one. Certificate export and checking in a separate controlled process provide the stronger separation.

13.2 Do not confuse an imported artifact with a replayed proof

The release artifact should carry the proof evidence needed for the chosen validation mode, not just a statement that an earlier build succeeded. A minimal fast mode may trust a pinned library and recheck only the new certificate. A stronger mode rechecks its reachable dependencies as well. Both modes should state the trusted starting environment.

Lean's official documentation describes fresh kernel replay and a stronger workflow that compares a proposed proof against a separately trusted statement, with exported evidence and external checking. Those mechanisms are relevant implementation components; Contour should integrate with them rather than invent a weaker substitute [13].

A generated `.lean` file is a useful interoperable artifact, but replaying it can invoke elaboration and tactics again. Search-independent replay should instead retain elaborated evidence or explicit, tightly specified proof constructions and compare their resulting type with the frozen statement. The ordinary source export remains valuable for inspection and migration, even when it is not the lowest-trust verification format.

13.3 Computation and the trust profile

Not all ways of obtaining a computation result have the same trust requirements. Kernel reduction and proof-producing reflection can fit a strict evidence policy. Native evaluation may rely on additional assumptions about the compiler and runtime, and its exact accounting is toolchain dependent. Lean's current documentation makes that distinction explicit [14].

Consequently a strict Contour profile should allow only evidence accepted under its declared foundation policy, and a native-computation profile should record its additional trust requirements rather than presenting them as indistinguishable. This is a policy distinction, not an argument against fast computation. Large certificates may reasonably use different profiles for interactive development and independent release checking, provided the difference is visible.

13.4 A proof lock file

A proposed `proof.lock` records the environment and choices that make the proof reproducible: toolchain and library revisions; the resolved statement; notation and instance selections; source-step identities; selected method versions and theorem instances; actual dependency closures; axiom inventory; and certificate hashes.

A hash establishes identity relative to an encoding; it does not establish mathematical correctness. The corresponding evidence still has to be checked. The encoding must include ordered binders, universe information, and semantic dependencies rather than relying on pretty-printed formulas as canonical keys.

There should be two build modes. **Explore** may run search, suggestions, or external workers and produce a new lock file. **Verify** consumes the locked evidence and performs no theorem search, model query, or network-dependent discovery. Verification can still perform the reduction required by type checking; "no search" does not mean "no computation."

A source edit invalidates the affected assertions and their dependents. An unrelated paragraph edit should not invalidate a certificate merely because its line number changed. A library change may preserve some semantic dependency closures and invalidate others. The initial implementation can conservatively invalidate an entire pinned environment; finer-grained reuse is an optimization, not a soundness requirement.

Proposition 13.1 (A limited replay-stability property)**.** *Suppose a closed certificate $p : T$ checks in an environment $\mathcal{E}$. Let $\mathcal{E}'$ preserve the declarations, universe constraints, and kernel-relevant definitions in the complete dependency closure of p and T, under the same kernel rules. Then rechecking the same certificate and target in $\mathcal{E}'$ yields the same typing judgment.*

Proof. Every constant lookup and reduction needed by the typing derivation has the same result under the stated preservation hypothesis. Reuse the original derivation, or induct over its typing and conversion steps. Declarations outside this closure do not occur in those steps. □

This is deliberately weaker than claiming that proof search is stable when the library grows. New simplification rules or search providers may change elaboration, search order, and generated evidence. Locked replay avoids that dependence; a fresh search does not.

14 A concrete implementation path

14.1 Begin inside Lean

The first implementation should be a Lean package with custom document syntax and elaborators, not a new kernel or an independent reimplementation of all mathematical expression parsing. Reuse Lean terms inside formulas, extending notation only where the intended meaning can be recorded precisely. A separate file extension and richer typesetting can follow once the semantics are stable.

The initial structural language needs only theorem blocks, scoped introductions, named assertions, calculations, case splits, induction, and explicit method calls. It should also support a local escape into ordinary Lean. Such an escape produces a term for one fixed obligation and passes through the same acceptance and assumption audit; it cannot mutate the theorem's meaning or silently add assumptions.

The core data model should retain source identities, typed contexts, exact targets, pending metavariables, dependency edges, accepted evidence, and explanation metadata. Separating these from the user-interface rendering prevents the editor from becoming the authority for proof status.

14.2 Suggested module boundaries

Component	Owns	Must not decide
Syntax and document layer	Scopes, source identities, mathematical presentation	Whether an unproved assertion is true.
Statement elaborator	Resolved constants, binders, instances, exact target	A different target because it is easier to prove.
Obligation service	Typed graph, dependency closure, transactional state	Acceptance of unchecked external evidence.
Method library	Transformation schemas and side-condition generation	New foundational inference rules.
Synthesis adapter	Translation, candidates, origin association, budgets	Truth of the original goal from fragment search alone.
Acceptance and export	Exact-type checking, assumption policy, replay artifacts	The author's intended informal meaning.
Editor and explanation layer	Navigation, expansions, diagnostics, proposed edits	Proof status based only on generated prose.

A completed obligation should be represented by a distinct type whose constructor is private to the acceptance layer. This is useful software architecture, although access control alone is not a substitute for independent checking. The exported artifact remains the logical authority.

14.3 First vertical slice

The most useful first milestone is not a large grammar. It is one theorem passing through the complete pipeline: fixed statement, structured source, generated obligations, checked completions, expandable explanations, and search-free replay.

The Lambert inequality is a suitable initial example. It exercises assumptions, ordered multiplication, positivity, normalization, a short calculation, and an exact conclusion. At this stage, every method can be an explicit wrapper around known Lean lemmas and tactics, with no theorem discovery at all. The test is whether the architecture preserves the mathematical structure and emits usable diagnostics when a sign hypothesis is removed.

The second slice should add finite-sum bounds and reindexing, using the binomial kernel example. This forces a principled representation-view mechanism. The third should add formal substitution, using the uniqueness proof, which tests side conditions that are not merely linear arithmetic. Pascal-tail induction then tests scoped motives, case splits, and reusable recurrence interfaces.

Only after these slices work should broad synthesis be connected. Otherwise a powerful prover can conceal errors in the language's obligation generation and make it hard to tell which feature is responsible for success.

14.4 Integrating Leant incrementally

Initially, expose a command that submits one selected local obligation to Leant and previews its checked candidates. The author can adopt one candidate, after which the accepted evidence is stored. This already makes synthesis useful without introducing hidden background discovery.

Next, allow a routine profile to call the structural engine automatically for narrowly specified classes of obligations, such as implication composition and conjunction assembly. Preserve unsupported and inconclusive outcomes. Tactic suggestions involving induction or new lemmas should remain explicit plan suggestions until their behavior is understood.

Later, add provider retrieval from the exact local environment, dependency-aware candidate ranking, and mathematical method constraints. Each extension should preserve original-goal checking and origin association. A more sophisticated rank-N or dependent synthesis engine improves coverage, but does not change the definition of a completed theorem.

14.5 Non-goals for the first implementation

The initial package should not promise unrestricted natural-language parsing, automatic discovery of arbitrary invariants, universal analytic automation, or automatic equivalence between every mathematical representation. It should not duplicate Mathlib's theorem corpus, and it should not depend on an online model to recheck existing proofs.

These scope choices leave room for a useful system. Most of the administrative friction identified in the four examples lies in a comparatively small set of repeatable operations. Building excellent support for those operations is a more concrete research program than placing an unconstrained synthesizer behind the word "obvious."

15 How to evaluate the proposal without fooling ourselves

15.1 Three baselines and honest accounting

The evaluation must compare at least three versions of each argument: the original Lean proof, a carefully refactored Lean proof, and the Contour version. They should use the same mathematical library interfaces and prove the same elaborated statement under the same assumptions.

Helpers must be counted honestly. Replacing fifty lines with a call to a newly introduced fifty-line lemma is useful abstraction, but it is not a fifty-line reduction in total formalization effort. Report both first-use cost and amortized reuse cost. Separate syntax savings from improvements caused by a new theorem, a better API, or a more powerful tactic.

Source lines alone are a poor primary metric because notation density and formatting can manipulate them. Better measures include the number of mathematical decisions visible to the reader, the number of explicit administrative operations, time to complete or repair a proof, comprehension of the argument, verification cost, and robustness under controlled changes. None of those measurements has been performed for this concept article.

15.2 Prevent theorem leakage

The target theorem already exists in ProveIt. An experiment that imports that theorem, or a later theorem implying it, can make a purported reconstruction trivial. The benchmark environment must stop before the target declaration, or remove the target and its downstream dependency closure from every retrieval and simplification channel.

It is not enough to blacklist the theorem's name. Equivalent aliases, previously generated certificates, helper lemmas that depend on the target, and search caches can leak the result. A benchmark harness should record the actual theorem dependencies of every accepted proof and verify that they belong to the allowed pre-target environment.

Use held-out modules or families of identities for generalization tests. The four examples in this article are design cases, not an unbiased test set: the proposed interfaces were deliberately motivated by them.

15.3 Measure both authoring and maintenance

Authoring experiments should record completion rates including failures, total interaction time, the number and nature of manual repairs, and the final proof's mathematical dependencies. Maintenance experiments should perturb notations, harmless variable names, lemma APIs, domain assumptions, and nearby library content.

Verification measurements should separate search time, elaboration time, certificate construction, kernel checking, and dependency replay. Record distributions and high-percentile behavior, not just a favorable example. A source that is short but unpredictably requires a huge search is not necessarily an improvement.

For readability, ask mathematically competent readers to identify the central idea, the role of each hypothesis, and the validity of a proposed modification. Compare refactored Lean, Contour, and an ordinary mathematical presentation without telling readers which design is expected to win. A language can be shorter yet harder to understand because too much reasoning has become implicit.

15.4 Ablations

Disable synthesis while keeping notation and method contracts; disable representation views while retaining synthesis; disable automatic side-condition inference; and replace contract-bound methods with unrestricted local search. These comparisons help identify whether a gain comes from the language structure, a particular automation engine, or simple library refactoring.

Also compare exploratory reconstruction with locked replay. A successful architecture should demonstrate that the latter does not depend on the availability of the original search worker. This is a reproducibility property, not a claim that verification will always be cheap.

15.5 Negative tests are part of the language specification

The following tests should accompany the positive corpus.

Perturbation	Required behavior
Multiply an inequality by a term of unknown sign	Report the missing sign condition or require an explicit case split.
Make a Lambert bound strict at $x = 0$	Reject the proposed proof; the strict claim is false there.
Use integer subtraction laws for unrestricted naturals	Require bounds or reject the transformation.
Reindex a sum with a non-injective map	Request multiplicity correction; do not accept a bijective reindexing certificate.
Cancel a nonzero zero divisor	Reject cancellation without a suitable cancellation proof.
Use formal-series associativity without required admissibility	Generate the missing substitution obligation.
Let an existential witness depend on a later binder	Reject the scope violation.
Solve two conjuncts with incompatible shared witnesses	Reject the merge.
Prove two pending assertions only from each other	Reject the unsupported cycle.
Exhaust a bounded synthesis search	Report an inconclusive outcome, not refutation.
Hide a missing proof in an imported dependency	Fail the transitive assumption audit.
Change notation so the displayed theorem means something else	Invalidate the frozen-statement match and show the semantic change.

A useful acceptance criterion is therefore not "the examples became short." It is "the examples became easier to author and understand, while controlled invalid variants were rejected for the right reasons and accepted proofs remained independently replayable."

16 Conclusion: a better unit of formal interaction

The reviewed code does not suggest that Lean's foundations are the main obstacle. It suggests a mismatch in granularity. A mathematician says "reindex the sum," "apply the logarithm," or "compose with the inverse." The implementation often requires a sequence of interval conversions, congruence applications, cast normalizations, and auxiliary side-condition proofs.

The proposed remedy is to make a *typed mathematical move* the unit of interaction. Its conclusion is fixed; its method is specified; its omitted conditions are generated; its evidence is checked; and its detailed realization can be expanded when needed. The source is concise because implementation details have a disciplined home, not because rigor has been removed.

Leant offers a useful model for the synthesis boundary: preserve the original problem, distinguish candidates from verified terms, carry provenance with evidence, and label failed search honestly. A human-scale proof language can use those capabilities without requiring the synthesis engine to understand every piece of mathematics hidden inside a dependent formula.

The immediate recommendation is a Lean-hosted prototype with four structural features—named assertions, calculations, scoped cases, and explicit induction—and two carefully specified method families: ordered-field transformations and finite-sum transformations. Add theorem-backed representation views, exact-target acceptance, and replay before broad automated discovery. Then use the formal-series example to test whether the architecture scales beyond arithmetic bookkeeping.

The long-term goal is not a system in which the mathematician writes less by saying nothing. It is a system in which the mathematician writes the argument, and the machine supplies the verifiable detail at the right level of abstraction.

A A compact grammar for the structural core

The grammar below specifies a minimal core, not every convenience phrase used in the worked examples. Expressions are delegated to an extensible, typed mathematical-expression parser with explicit notation profiles. Higher-level phrases such as `calculate`, `the stated chain`, and bundled-structure binders lower to this core.

```
document    ::= declaration*
declaration ::= "theorem" name binder* ":" formula proof
proof       ::= "proof" statement* "end"
statement   ::= take | assume | let | have | choose
              | suffice | conclude | cases | induction | calculation

take        ::= "take" name ":" expression
assume      ::= "assume" name ":" formula
let         ::= "let" name [":" expression] ":=" expression
have        ::= "have" name ":" formula justification
choose      ::= "choose" name ":" expression
                "with" name ":" formula justification
suffice     ::= "suffices" formula justification
conclude    ::= "conclude" (formula | "target") justification
justification ::= "by" method ["using" "[" fact-list "]"]
                | proof
method      ::= method-call (";" method-call)*
method-call ::= name ["(" argument-list ")"]

cases       ::= "cases" formula branch+ "endcases"
branch      ::= "case" pattern ":" statement*
induction   ::= "induct" name
                ["keeping" name-list]
                ["generalizing" name-list]
                branch+ "endinduct"
calculation ::= "calc" expression calculation-edge+ "endcalc"
calculation-edge ::= relation expression justification
relation    ::= "=" | "<=" | "<" | registered-relation
```

A layout-sensitive surface can infer the block terminators. The abstract syntax retains them. Mathematical profiles can add notation for sums, derivatives, coefficient extraction, or composition, but cannot add unchecked inference rules. A profile records the formal meaning of each notation and the theorems underlying each method.

The `choose` command has two context-sensitive uses: constructing a witness for an existential goal and eliminating an established existential premise. The elaborator distinguishes them from the typed goal and justification, and introduces the correct constructor or eliminator. An ambiguous use is rejected rather than interpreted by whichever search happens to succeed.

B An example of a structured method certificate

For the scaled tangent inequality in Section 5, a method certificate could retain the following tree. This is a proposed interchange structure, not an implementation-specific Lean serialization.

```
node: scaled-tangent
context:
  w : Real
  tangent : 1-w <= exp(-w)
target:
  exp(w)*(1-w) <= 1
method:
  normalize-after-positive-multiplication
children:
  positivity:
    target: 0 <= exp(w)
    proof: nonnegative_of_positive(exp_positive(w))
  multiplication:
    target: exp(w)*(1-w) <= exp(w)*exp(-w)
    proof: ordered_multiplication(tangent, positivity)
  normalization:
    target: exp(w)*exp(-w) = 1
    proof: exponential_addition_and_zero
assembly:
  rewrite_right_endpoint(multiplication, normalization)
```

The three children are checked at their stated types, and the assembly constructor is checked as well. A bad positivity claim, reversed multiplication inequality, or incorrect normalization fails without having to trust the method worker. The final type is compared with the frozen target.

For explanatory fidelity, the root must also match the advertised method schema: the multiplication child must be built from the referenced tangent fact and the selected multiplier. This syntactic and typed structure is checkable. It does not establish that the multiplier was an inspired choice or that the argument is the shortest possible proof.

The release artifact should use exact elaborated expressions and declaration references, not the informal names in this diagram. The readable tree is a presentation derived from those references. A separate export can materialize the corresponding Lean expression or independently checkable serialized certificate.

C Repository inspection manifest

The repositories were inspected through their current GitHub content, then references were pinned to the following revisions:

```
ProveIt: 24ce8bd743eaab64a91ce90725ea00f498d319d2
Leant:   c36adf114035fb2fba6c276b63944cc613b31668
```

The four ProveIt modules were read in full. Their common directory is

```
Analysis/FabiusFunction/Lean/FabiusFunction/
```

`BinomialInversion.lean`. Blob identifier:

```
c6b0fe63a4faf8708b71e9b93aa2ce7406464d84
```

Principal inspected declarations: `sum_Icc_neg_one_pow_choose_mul_choose`, `sum_range_neg_one_pow_sub_mul_choose`, `binomial_inversion`, and `binomial_inversion_ring_iff`. The review also used the direct reverse-orthogonality application and the cast-transport theorem.

`LambertWElementaryBounds.lean`. Blob identifier:

```
fdc8fc743ed83f6f4ba3371e09fca1f6c960dab9
```

Principal inspected declarations: `exp_le_one_add_mul_exp_self`, `le_log_one_add_mul_exp`, `mul_exp_div_one_add_le`, `principalLambertW_nonneg`, and `principalLambertW_elementary_bounds`, together with the strict variants.

`LagrangeInversionUniqueness.lean`. Blob identifier:

```
b0461940874d53d126193093b5bdea933690f0ab
```

Principal inspected declarations: `eq_solution_of_eq_X_mul_subst`, `existsUnique_solution`, `existsUnique_of_isUnit_constantCoeff`, `coeff_jacobian_mul`, and `coeff_subst_alt`.

`PascalTailIdentity.lean`. Blob identifier:

```
4581b581fc501650ab1e35585500f20d05db8cd5
```

Principal inspected declarations: `tailCount`, `tailCount_succ_add`, `sum_choose_mul_two_pow_add_tailCount`, and the subtraction and upper-bound corollaries.

For Leant, the inspection covered the README overview and synthesis discussion, including pinned lines 340–520; `src/Leant/Synth/Fragment.hs`, lines 1–200; and `docs/synth-internals.md`, lines 1–240. The corresponding blob identifiers are:

```
README.md:
  2f26dd747e76ccfcf0850d177c6516b65a572363
src/Leant/Synth/Fragment.hs:
  ab523ffd5e93a0ddc8c5d893a79e032562319aa1
docs/synth-internals.md:
  021c9385368a7087e6141f03a18f27e4adafa188
```

Descriptions of Leant's execution behavior are grounded in those inspected sources and documentation. This study did not rerun Leant's reported test suites, rebuild ProveIt, or implement the proposed language. The bibliography links the exact repository revisions, while external references supply context for existing proof languages, automation, and Lean's verification model.

References

[1] Vladimir Reshetnikov and contributors. *ProveIt: BinomialInversion.lean*. Revision `24ce8bd743ea`, 2026. Pinned source file.

[2] Vladimir Reshetnikov and contributors. *ProveIt: LambertWElementaryBounds.lean*. Revision `24ce8bd743ea`, 2026. Pinned source file.

[3] Vladimir Reshetnikov and contributors. *ProveIt: LagrangeInversionUniqueness.lean*. Revision `24ce8bd743ea`, 2026. Pinned source file.

[4] Vladimir Reshetnikov and contributors. *ProveIt: PascalTailIdentity.lean*. Revision `24ce8bd743ea`, 2026. Pinned source file.

[5] Vladimir Reshetnikov and contributors. *Leant: README*, especially interactive proving and automatic term synthesis. Revision `c36adf114035`, 2026. Pinned documentation.

[6] Vladimir Reshetnikov and contributors. *Leant: src/Leant/Synth/Fragment.hs*, inspected lines 1–200. Revision `c36adf114035`, 2026. Pinned source file.

[7] Vladimir Reshetnikov and contributors. *Leant: Synthesis internals*, inspected lines 1–240. Revision `c36adf114035`, 2026. Pinned design documentation.

[8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, LNCS 12699, pp. 625–635, 2021. DOI: 10.1007/978-3-030-79876-5_37.

[9] The Lean development team. *The Lean Language Reference*. Online reference, consulted 5 September 2026. Official reference manual.

[10] The Lean development team. *Elaboration and Compilation*, in the Lean Language Reference. Consulted 5 September 2026. Official chapter.

[11] The Lean development team. *Propositions*, in the Lean Language Reference, The Type System. Consulted 5 September 2026. Official chapter.

[12] The Lean development team. *Inductive Types*, in the Lean Language Reference, The Type System. Consulted 5 September 2026. Official chapter.

[13] The Lean development team. *Validating a Lean Proof*, in the Lean Language Reference. Consulted 5 September 2026. Official validation guidance.

[14] The Lean development team. *Frequently Asked Questions*, especially the trusted code base, proof objects, and totalized operations. Consulted 5 September 2026. Official FAQ.

[15] Grzegorz Bancerek et al. The role of the Mizar Mathematical Library for interactive proof development in Mizar. *Journal of Automated Reasoning* 61, pp. 9–32, 2018. DOI: 10.1007/s10817-017-9440-6.

[16] Makarius Wenzel and the Isabelle project. *Isabelle/Isar*. Project description and documentation, consulted 5 September 2026. Official Isar page.

[17] Adrian De Lon et al. The Isabelle/Naproche natural language proof assistant. In *Automated Deduction—CADE 28*, LNCS 12699, pp. 614–624, 2021. DOI: 10.1007/978-3-030-79876-5_36.

[18] Patrick Massot. Teaching mathematics using Lean and controlled natural language. In *ITP 2024*, LIPIcs 309, pp. 27:1–27:19, 2024. DOI: 10.4230/LIPIcs.ITP.2024.27.

[19] Jelle Wemmenhove et al. Waterproof: Educational software for learning how to write mathematical proofs. *Electronic Proceedings in Theoretical Computer Science* 400, pp. 96–119, 2024. DOI: 10.4204/EPTCS.400.7; arXiv:2211.13513v2.

[20] Freek Wiedijk. Formal proof sketches. In *Types for Proofs and Programs, TYPES 2003*, LNCS 3085, pp. 378–393, 2004. DOI: 10.1007/978-3-540-24849-1_24.

[21] Leslie Lamport. How to write a 21st century proof. *Journal of Fixed Point Theory and Applications* 11, pp. 43–63, 2012. DOI: 10.1007/s11784-012-0071-6.

[22] Jannis Limperg and Asta Halkjær From. Aesop: White-box best-first proof search for Lean. In *Certified Programs and Proofs, CPP 2023*, 2023. DOI: 10.1145/3573105.3575671.

[23] The Lean development team. *The grind tactic*, in the Lean Language Reference. Consulted 5 September 2026. Official chapter.

[24] Albert Q. Jiang et al. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. 2022, revised 2023. arXiv:2210.12283.